WEEK 11 ● LECTURE 21

Polyphase Decomposition and Efficient Decimation

Reorganize the same FIR computation so we never calculate samples that will be discarded

$$H(z) = \Sigma\, z^{-k}\, E_k(z^M)$$

$\downarrow M$ + polyphase = low-rate arithmetic

Oppenheim & Schafer §4.7–4.8 | Raschka: convolutional architectures / efficient ML

Roadmap and learning objectives

75 minutes • decomposition → Noble identity → efficient decimator → ML structured parallelism

- Decompose an FIR impulse response into M polyphase subsequences.
- Explain exactly where the M-fold decimation savings comes from.
- Use the first Noble Identity correctly—and know when it does not apply.
- Distinguish polyphase time-phase partitioning from grouped/channel convolution in ML.

0–20 min

Polyphase concept and index bookkeeping

20–55 min

Naive vs efficient decimation + Noble identity

55–75 min

Grouped convolutions / edge efficiency

01

The polyphase concept

Partition FIR taps by their index modulo M. Nothing is approximated; this is an exact algebraic rearrangement.

Start with one ordinary FIR filter

For h[n], separate taps whose indices have the same remainder modulo M.

$$e_k[r] = h[rM + k], \quad k = 0,1,\ldots,M-1$$

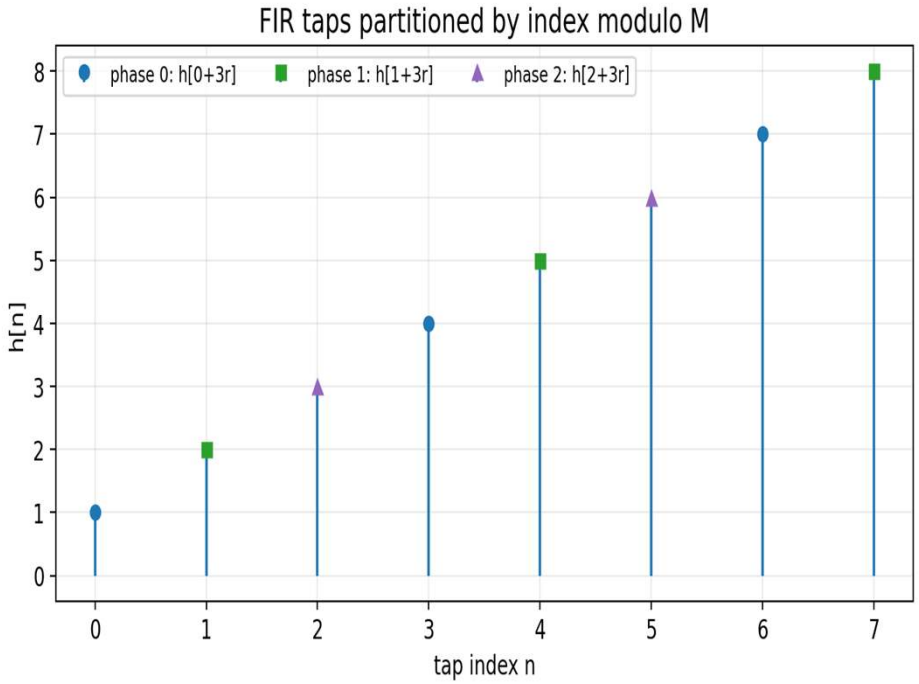
$$E_k(z) = \Sigma_r \, e_k[r] \, z^{-r}$$

Interpretation

Phase k contains the FIR taps at indices k, k+M, k+2M, ...

Why this helps

The same phase structure lines up naturally with keeping every Mth output sample.



Polyphase decomposition in the z-domain

Collect the M residue classes of tap indices.

$$H(z) = E_0(z^M) + z^{-1}E_1(z^M) + \ldots + z^{-(M-1)}E_{M-1}(z^M)$$

$$H(z) = \sum_{k=0}^{M-1} z^{-k}\, E_k(z^M)$$

- Each E_k operates on every Mth FIR coefficient.
- The delays z^{-k} restore the original tap offsets when branches are recombined.
- No coefficients are changed and no output samples are changed.

Memory aid

Polyphase = "group coefficients by n mod M, then express the slow-time branch with z^M."

CONCEPT CHECK

For M = 4, which FIR taps belong to phase E_2?

CONCEPT CHECK

For M = 4, which FIR taps belong to phase E_2?

Good answer

Tap indices n = 2, 6, 10, 14, … because they satisfy n = 4r + 2. Equivalently, $e_2[r] = h[4r+2]$.

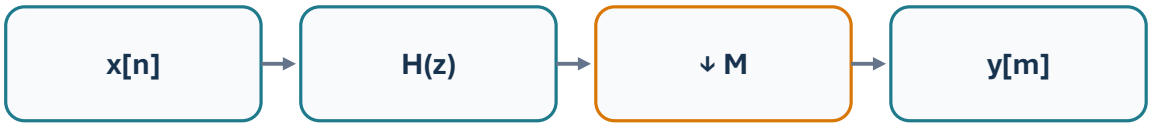
02

Efficient decimation and Noble identities

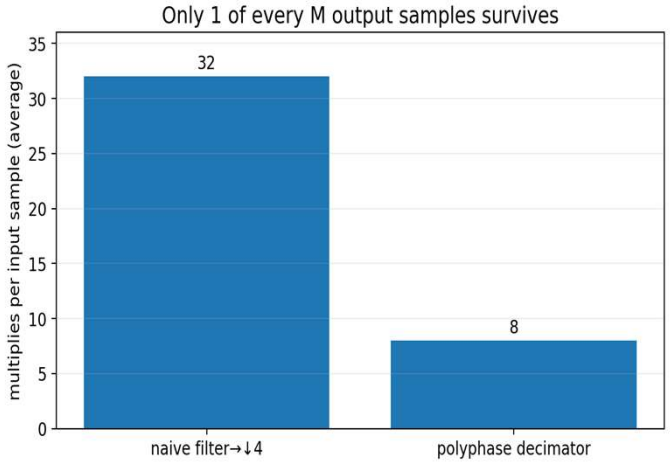
Move computation to the slow side of the rate changer—but only when the structure permits it.

The naive decimator wastes most FIR outputs

Filter at F_s, then ↓M keeps only v[0], v[M], v[2M], ...



$$v[n] = \Sigma_k\, h[k]x[n-k] \quad \rightarrow \quad y[m] = v[mM]$$



Cost intuition

An N-tap FIR evaluates about N multiplies for every input sample, even though only 1/M of the outputs remain.

Target

Keep exactly the same y[m], but evaluate only the convolution sums associated with n = mM.

First Noble Identity: the precise statement

A rate changer commutes with a filter only after the filter has the required z^M structure.



What changes

On the right, G(z) runs at the lower sampling rate F_s/M.

What does NOT work

You cannot commute ↓M through an arbitrary G(z) and expect the same system.

$$G(z^M) \rightarrow \downarrow M \quad \equiv \quad \downarrow M \rightarrow G(z)$$

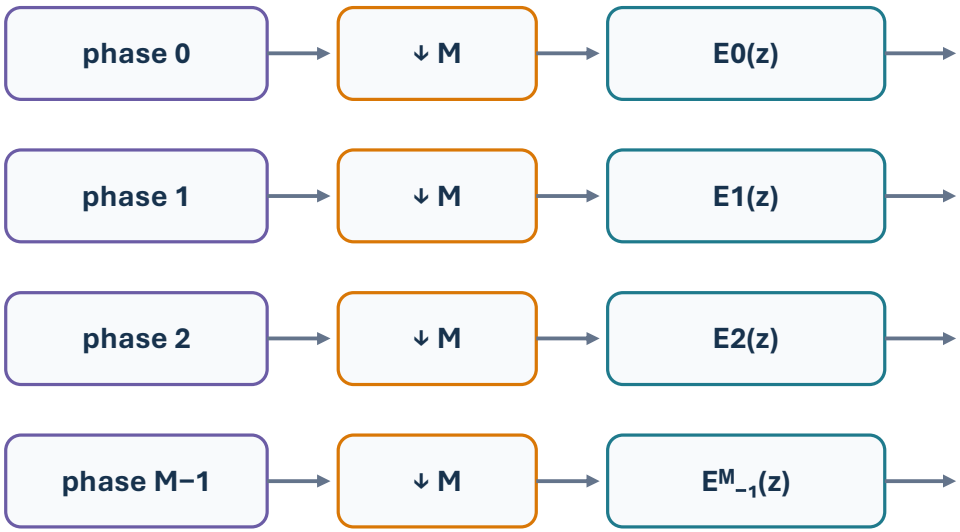
Why polyphase matters

Every branch in $H(z)=\sum z^{-k}E_k(z^M)$ contains exactly the form needed by the Noble Identity.

Efficient polyphase decimator

Decompose H(z), then move each E_k through the downsampler so it runs at the output rate.

$$y[m] = \Sigma_k \Sigma_r e_k[r] \cdot x[(m-r)M - k]$$



Key result

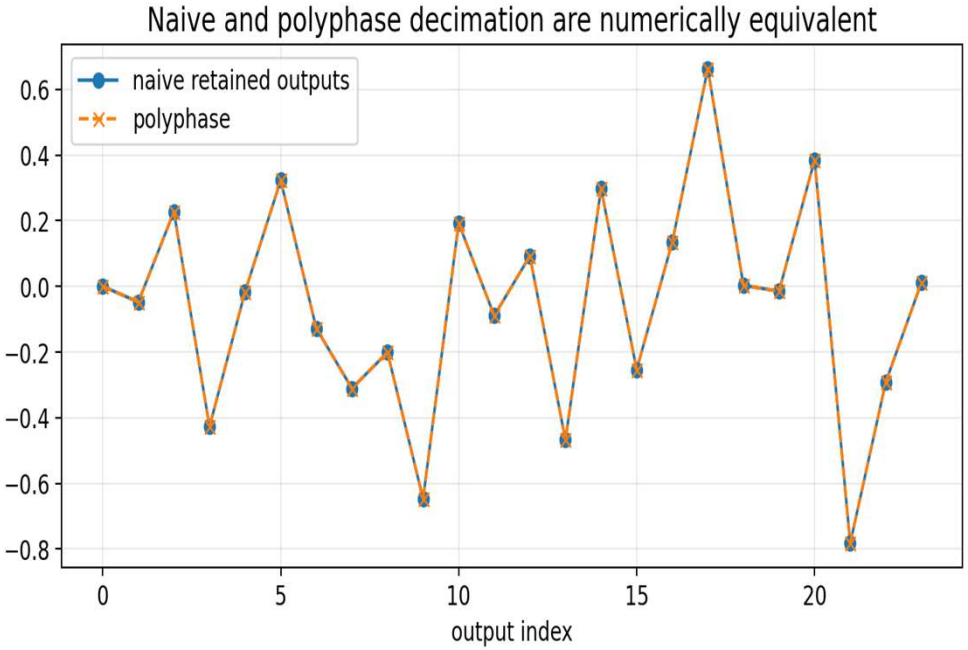
All E_k filters execute at F_s/M, not F_s.

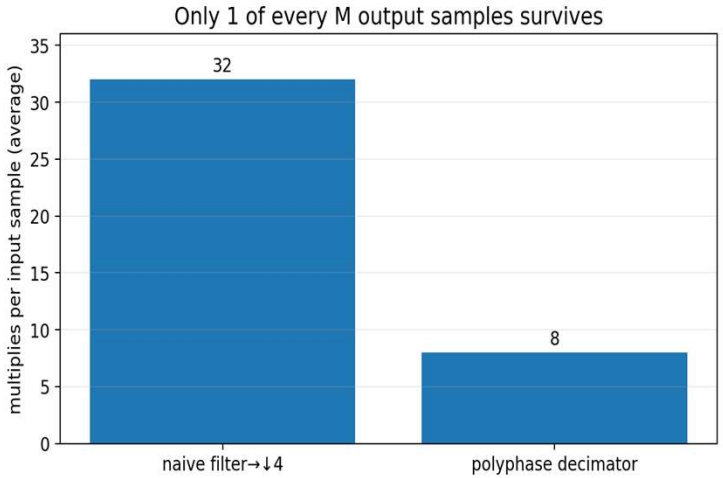
Same math

Branches are phase-selective views of the same original convolution.

Where the M-fold savings comes from

The number of multiplies per retained output is not smaller; the number of unnecessary outputs is.





naive ≈ $N \cdot F_s$ mult/s

polyphase ≈ $(N/M) \cdot F_s$ mult/s

Ratio

Ideal arithmetic saving ≈ M for long FIRs and steady-state streaming.

CONCEPT CHECK

An N = 48 tap FIR decimator reduces the rate by M = 6. Roughly how many FIR multiplications per input sample are needed after polyphase reorganization?

CONCEPT CHECK

An N = 48 tap FIR decimator reduces the rate by M = 6. Roughly how many FIR multiplications per input sample are needed after polyphase reorganization?

Good answer

About N/M = 8 multiplications per input sample on average, versus about 48 for filtering every full-rate output first.

03

ML context: grouped convolutions

Both ideas partition a large computation into structured independent branches—but along different axes.

Grouped Conv1d: channel partitioning

PyTorch groups restrict which input channels can contribute to which output channels.

$$\mathrm{Conv1d}(C_{in}, C_{out}, \mathrm{kernel_size}=K, \mathrm{groups}=G)$$

Standard convolution

Every output channel can mix all input channels.

Grouped convolution

Channels are split into G independent convolution groups.

Depthwise convolution

$G = C_{in}$: each channel is filtered independently before later mixing.

- Fewer cross-channel connections → fewer parameters and MACs.
- A following 1×1 / pointwise convolution can restore channel mixing.
- This structured factorization is a common edge-deployment idea.

Polyphase versus grouped convolution

Useful analogy: partition → parallel low-cost branches → recombine. Important difference: what is partitioned?

Polyphase DSP

Partitions sample/tap phases modulo M.

Goal: exact multirate equivalence while moving arithmetic to a lower sample rate.

Branches are determined by the rate-change factor.

Grouped CNN

Partitions feature channels.

Goal: constrain connectivity and reduce learnable computation.

Groups are an architectural choice and weights are learned.

shared principle: exploit structure instead of computing a dense operation

Coding demo: prove the equivalences

Use the notebook to compare outputs and perturbation paths.

- Decompose h[n] into $e_0,\ldots,e^M_{-1}$ and reconstruct it exactly.
- Compare filter→↓M with scipy.signal.upfirdn(..., down=M).
- Numerically verify the First Noble Identity for $G(z^M)$.
- Perturb one input channel of a grouped Conv1d and observe that only outputs in its group change.

Prediction first

Ask students which outputs should match before running each cell.

Debugging habit

If a Noble-identity test fails, check whether the filter actually has z^M spacing.

Notebook

Week7_Polyphase_DSP_ML_Demos.ipynb

Lecture 21 wrap-up

Polyphase does not change the filter—it changes where and when the arithmetic happens.

- $H(z)=\Sigma\, z^{-k}E_k(z^M)$ groups FIR taps by phase.
- A naive decimator computes outputs that $\downarrow M$ immediately discards.
- The First Noble Identity moves E_k filters to the low-rate side because they arise as $E_k(z^M)$.
- Ideal arithmetic savings are approximately M.
- Grouped convolution is a structural parallelism analogy, not the same decomposition.

EXIT QUESTION

Why is it incorrect to say “a downsampler can always be moved before a filter”?

Good answer

The Noble Identity only permits the move for the special $G(z^M)$ structure; arbitrary filters do not commute with $\downarrow M$.

References and instructor resources

Week 11 • Lecture 21

- Oppenheim & Schafer, Discrete-Time Signal Processing, §4.7: polyphase structures and multirate implementations.
- Raschka et al., Machine Learning with PyTorch and Scikit-Learn: convolutional neural-network architectures.
- Accompanying notebook: Week7_Polyphase_DSP_ML_Demos.ipynb.
- Homework and weekly quiz: Week7_Polyphase_Homework_Quiz_Solutions.docx.